CS 498: Machine Perception
Final Project — Track 1
Multi-View 3D Semantic Occupancy Prediction

Fall 2026

Base credit	20 points
Work mode	Individual
Release	Wednesday, November 4, 2026
Due	December 9, 2026, 11:59 p.m. America/Chicago
Competition	One Kaggle ranking over all 10 Test houses

Non-AI-assisted assignment. Do not use generative-AI systems, coding agents, AI code completion, or AI-generated report content. Course-authorized accessibility tools are the only exception.

1 What you will build

Can three photographs reveal more than visible surfaces? Build a feed-forward system that uses overlapping RGB images, optional depth, and calibrated camera geometry to infer 3D semantic occupancy, including eligible hidden parts of objects in view. Each voxel is free space or one of nine occupied semantic classes.

The three images are *one independent sample*, not video frames: one camera center, headings $-45°, 0°, +45°$, and 90° field of view per image. Adjacent views overlap by 45°. No temporal map or other sample is available at inference.

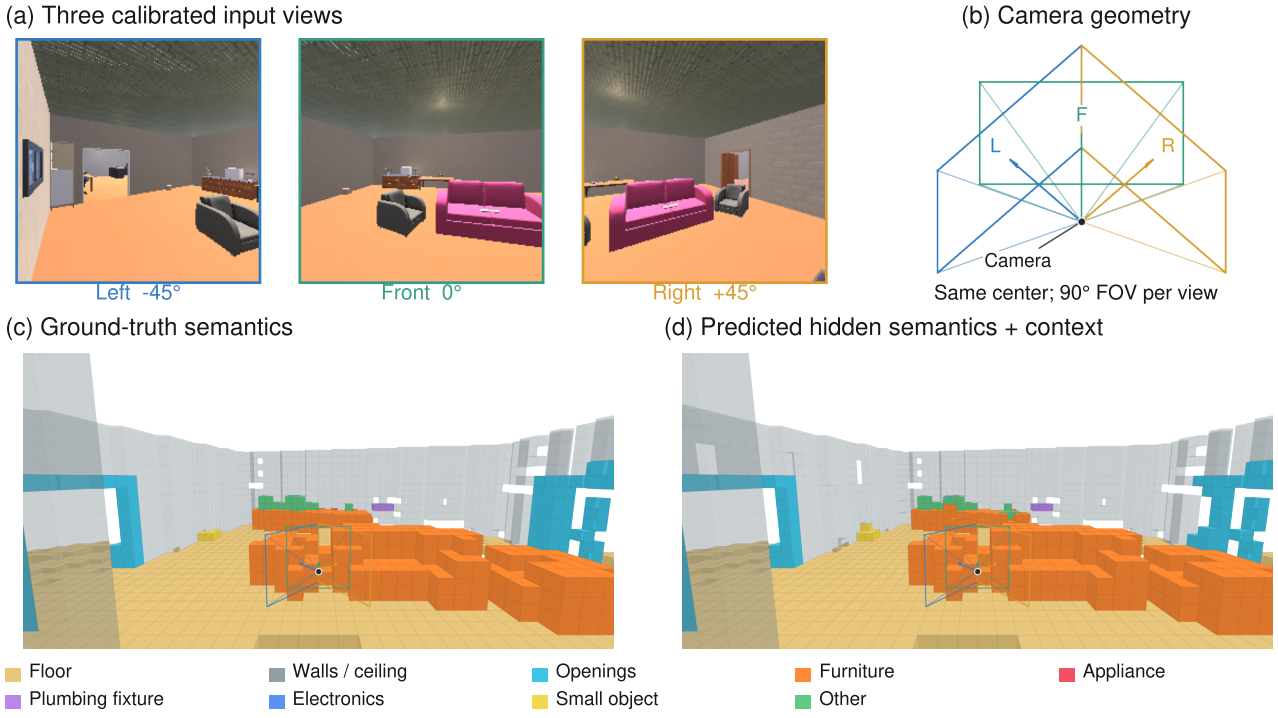


Figure 1: A Val example with co-located cameras, color-matched to the input borders. Voxel colors denote semantic classes, not visibility. Right: real predictions on hidden cells with shared GT context; walls are translucent for clarity. GT context and masks are display-only, never inference inputs.

All data are provided, rendered with ProcTHOR and AI2-THOR [1, 2]. You may complete the eight baseline blocks and improve that method, or develop your own algorithm through the common interface. No scene generation or simulator installation is required.

2 Deliverables and grading

Category	What is evaluated	Points
Model and system design	correctness, technical soundness, rationale, implementation quality, appropriate validation use	8
Report writing	clarity, reproducibility, related work, experimental design, limitations	8
Analysis and visualization	quantitative metrics, controlled comparisons, error analysis, qualitative results, simulator-data demonstration	4
Total		20
Competition bonus	eligible top-three finishers after reproduction audit	0–2 extra

The report should explain a technically justified method, not merely quote a leaderboard score. A policy-compliant, reproducible entry matching or exceeding the verified reference score **0.65135** earns all 8 Model and system design points. Compare at Kaggle's displayed five-decimal precision, not unpublished extra decimal places. This fixed course benchmark does not require a leaderboard participant entry. Other entries are assessed on the criteria above; report and analysis credit remain separate.

The top three final finishers may earn 1–10 percentage points of bonus, equivalent to +0.2 through +2.0 on the 20-point project scale, depending on final performance and reproduction review. After the competition ends, eligible winners must publish the exact audited code in GitHub and learned weights in Hugging Face. Training-free methods declare that no weights exist. Do not publish these before the deadline.

Upload the generated `cs498-track1-YOUR_NETID.zip` and its separate `package_manifest.json` to the LMS. The ZIP contains:

```
student_submission/        # final code, method.json, dependencies, weights/
report.pdf
outputs/evaluation/val/summary.json
outputs/report/            # four figures + evidence_manifest.json
outputs/kaggle/submission.csv
outputs/kaggle/submission_runtime.json
```

Separately upload only `submission.csv` to Kaggle. Do not include datasets, credentials, caches, hidden labels, or another student's work in either submission.

3 Setup, data, and workflow

The supported paths are **Ubuntu with an NVIDIA GPU** and **Google Colab**. Colab is the fallback for students without a suitable local GPU and is recommended for **Mac and Windows** users; native platform execution is not required. Follow `SETUP.md` before the commands below. It provides isolated Ubuntu installation, tested dependencies, report tools, storage guidance and troubleshooting. The runnable Colab entry is `notebooks/track1_colab.ipynb`.

Use Python 3.11–3.12 (the Kaggle SDK requires 3.11+) and a CUDA-capable GPU for full training. The report requires `latexmk` or `pdflatex`. Run commands from the released project directory. All provided commands use `python run.py`; add `--help` to see options. Your editable method is in `student_submission/`; provided helpers are organized under `cs498_track1/`. Leave the provided tools unchanged.

```
python -m pip install -r requirements.txt
python run.py doctor --require-gpu --require-latex
python -m pytest -q tests
python run.py download --split train --houses train_house_000
python run.py download --split val --houses val_house_000
python run.py train --smoke
```

The untouched starter is runnable but intentionally noncompetitive: eight focused checks are `XFAIL`, while infrastructure tests pass. Editing a block activates its correctness check. Do not alter tests to hide failures. The smoke command checks training plumbing; placeholder predictions do not establish a correct implementation. Smoke artifacts stay in `outputs/train/smoke/`, separate from final submission weights.

Split	Houses	Samples per house	Total samples
Train	200	150	30,000
Val	10	15	150
Test	10	15	150

Each sample contains three RGB images, three depth images, intrinsics, three camera poses, and a front-camera anchor pose. Train/Val additionally include prepared labels and masks on the canonical output grid. Test contains inputs only. Semantic and instance images are not inference inputs and are not required by the starter.

The downloader uses pinned Hugging Face data, SHA-256 checks, and safe extraction into `data/triview/`. Subset downloads work without other splits. Its manifest reports archive sizes; budget for both cached archives and extracted data, plus model outputs.

After completing the blocks:

```
python run.py download --split train
python run.py download --split val
python run.py train                # at most 15 epochs
python run.py evaluate             # all 150 Val samples
python run.py figures
```

The default optimizer is AdamW with learning rate 3×10^{-4}, 1000-step warmup, cosine decay, batch size 32, mixed precision, and gradient clipping. You may use `--batch-size 8` or `--workers 0` when resources require it. `--rgb-only` disables depth. The reference architecture has 1.68M parameters. A full 15-epoch release check on an RTX 4080 took about 53 minutes, excluding downloads and the initial class-histogram scan. This is not a Colab/T4 timing guarantee; allow additional time for validation, experiments and reporting.

On Colab, keep code, weights, recovery checkpoints, outputs and the report in Drive, but download-/extract data on local runtime disk. A saved notebook does not save Python-file edits or local runtime data. Free GPU allocation and session duration are not guaranteed. After reconnection, reinstall runtime tools, reuse your saved project without overwriting it, and redownload the same pinned data locally.

Training can resume from a *completed epoch*; an interrupted epoch is repeated:

```
python run.py train --device cuda --epochs 15 --batch-size 8 --workers 0 \
  --resume outputs/train/checkpoint_last.pt
```

Use the original total epoch target, batch size, device, Torch version and unchanged code/data. Recovery restores optimizer, learning rate, random state and historical best weights, not just the last model. Read `SETUP.md` for the complete new-session and recovery sequence. Select a real GPU before full training; CPU is for smoke/interface checks. The notebook also explains editing the actual student files, report compilation, and downloading the CSV, ZIP and separate manifest.

4 Input, output, and supervision contracts

Camera coordinates are x-right, y-down, z-forward. `T_world_camera` maps camera coordinates into world coordinates. Depth is optical-axis z depth in metres, not Euclidean ray length. The local output frame uses x-right, y-up, z-forward relative to the front camera.

The fixed output bounds are $x \in [-10, 10)$ m, $y \in [-1.75, 1.75)$ m, and $z \in [0, 10)$ m at 0.25 m resolution. Arrays are `(z,y,x)`, with shape `(40,14,80)`. These bounds define the benchmark's prediction region; targets outside it are not scored. Cells outside eligible frusta or supervision support are ignored, not automatically free.

ID	Class	ID	Class
0	free	5	appliance
1	floor	6	plumbing fixture
2	wall / ceiling	7	electronics
3	opening	8	small object
4	furniture	9	other occupied

Unknown -1 is an ignore label in supervision, never a legal prediction. At inference, return a full grid of labels 0–9 without reading a target or output mask.

The target policy `three_view_occlusion_room_v1` uses the union of the three views. Directly measured free space and surfaces form observed evidence. Completion support may extend behind the first visible surface through that *same instance*, not through a distinct hidden instance. Unobserved supervision is restricted to the camera's current room; geometry directly seen through an opening remains an observation. The task does not ask you to reconstruct arbitrary unseen rooms.

Prepared targets are generated from the original 10 cm world labels. Source cells are transformed into the local frame and grouped into 25 cm cells. Occupied labels take priority over free labels; occupied semantics use the most frequent occupied class. A canonical cell is completion-supervised when at least half of its eligible source cells are hidden. Its label then uses hidden source cells only, avoiding leakage from observed surfaces into the hidden target. All this preprocessing is provided. Some samples have no eligible hidden cells after aggregation; they contribute no confusion counts, but remain required prediction rows. Every house has scored cells.

5 Baseline implementation

For the baseline route, complete eight functions in `semocc/` inside `student_submission/`. Other functions are provided. The architecture combines geometric feature lifting and a compact 3D encoder–decoder, drawing on established reconstruction and U-Net ideas [3, 4].

File	Required function
`model.py`	`project_voxel_centers`
	`TriViewLiftFuseUNet._project_and_sample`
	`TriViewLiftFuseUNet.fuse_views`
	`SpatialUNet3d.forward`
	`factorized_logits`
`learning.py`	`completion_loss`
	`class_weights_from_histogram`
	`metrics_from_confusion`

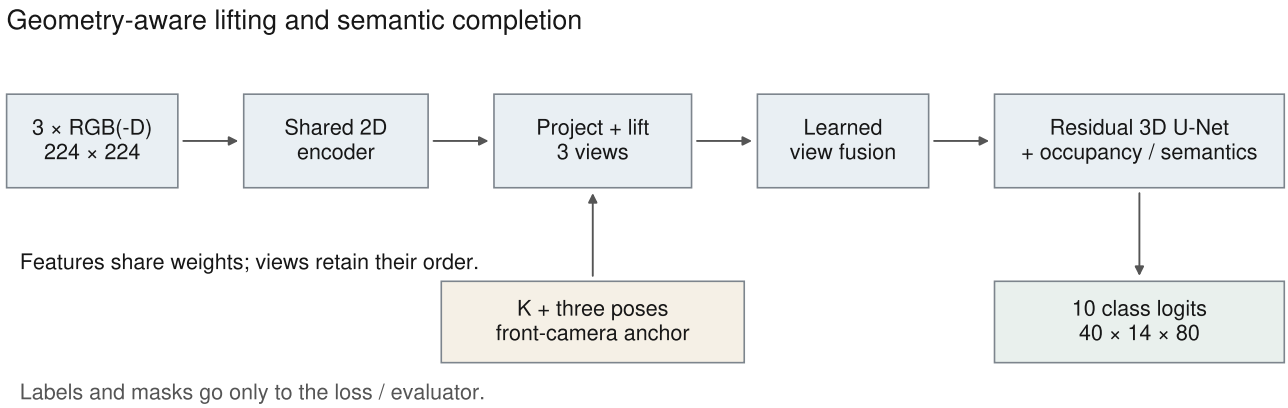


Figure 2: The provided architecture. Shared 2D features are projected into the same local 3D lattice, fused in fixed view order, and completed with a residual 3D U-Net. Only sensor inputs and calibration enter the network. Labels and masks enter the training loss and evaluator separately.

5.1 Project local voxel centers

`project_voxel_centers` receives local centers $\mathbf{q} = (x, y, z)$, original-image intrinsics, and camera/anchor poses. With $D = \mathrm{diag}(1, -1, 1, 1)$, transform homogeneous local points into view v:

$$\widetilde{\mathbf{p}}_v = T_{\mathrm{world},v}^{-1} T_{\mathrm{world,anchor}} D \widetilde{\mathbf{q}}.$$

Project $u = f_x p_x / p_z + c_x$, $v = f_y p_y / p_z + c_y$. A valid point has $p_z > 0.05$ and lies inside the original image bounds. For `grid_sample(...,align_corners=False)`, use normalized coordinates $g_x = 2(u+0.5)/W - 1$ and $g_y = 2(v+0.5)/H - 1$. Return the grid `[B*3,N,1,2]`, validity `[B,3,N]`, and camera z depths. Clamp the denominator for numerical safety without marking behind-camera points valid.

5.2 Lift features and fuse views

The supplied shared encoder maps `[B,3,C,H,W]` to three feature maps with $F = 24$ channels. In `_project_and_sample`, sample each feature map using the projected grid and zero invalid projections. With depth, sample the depth map at the same grid and append three channels per view: valid depth-projection indicator, signed depth residual $(d - p_z)/0.5$ clipped to $[-1, 1]$, and surface likelihood $\exp(-|d - p_z|/0.175)$. Invalid depth has zero residual and likelihood. With RGB only, retain the projection-valid indicator and use zero residual/likelihood.

Concatenate views in left/front/right order, reshape to the local volume, and append the three provided normalized local-coordinate channels. Thus the lifted tensor has $3(F + 3) + 3 = 84$ channels. Do not reduce away view identity. `fuse_views` applies the declared `view_fusion` module, which maps these 84 channels to 24 learned channels.

5.3 3D encoder–decoder and output heads

In `SpatialUNet3d.forward`, apply `stem`, `encoder1`, `down1`, `encoder2`, `down2`, and `bottleneck`. Interpolate to each matching skip resolution, concatenate along channels, apply `decoder2` then `decoder1`, and finish with `head`. Use trilinear interpolation with `align_corners=False`; interpolate to the skip's exact shape because the vertical dimension is not divisible by four.

The head gives one occupancy logit o and nine conditional semantic logits s_c. In `factorized_logits`, return ten ordinary class logits

$$\ell_0 = 0, \qquad \ell_c = o + \log \mathrm{softmax}(\mathbf{s})_c, \quad c = 1, \ldots, 9.$$

Then total occupied probability is $\sigma(o)$. Return logits, not probabilities.

5.4 Loss and class balance

Implement semantic cross entropy on valid completion cells, an auxiliary occupied/free loss on the same cells, and observed consistency:

$$L = L_{\text{semantic}} + 0.5L_{\text{occupancy}} + 0.1L_{\text{observed}}.$$

Cross entropy uses class weights, `ignore_index=-1`, and `reduction="none"`. The binary occupied logit is $\log \sum_{c=1}^{9} e^{\ell_c} - \ell_0$; its target is $y > 0$. Consistency uses `output_mask & ~completion_mask & (labels >= 0)`. For any empty loss domain, return a differentiable zero term rather than taking an empty mean. Return the total scalar and detached values named `semantic`, `occupancy`, and `observed`.

For `class_weights_from_histogram`, absent classes get zero. Present classes use inverse-square-root frequency; normalize their weights to mean one, clip to `maximum`, then normalize present weights to mean one again.

5.5 Metrics and validation

`metrics_from_confusion` takes a matrix indexed `[target,prediction]`. For class c, $\text{IoU}_c = C_{cc}/(\sum_j C_{cj} + \sum_i C_{ic} - C_{cc})$. Average only over classes with nonzero target support, *including free*. Accuracy is total diagonal over total count. Occupied IoU combines classes 1–9 into one occupied category, regardless of semantic agreement.

For zero class union, return class IoU zero. A zero-total confusion matrix gives zero accuracy and mean IoU; zero occupied union gives zero occupied IoU. Return class support counts as well as per-class IoUs.

Canonical evaluation pools all 15 samples within each house before computing metrics, then averages house scores equally. `python run.py evaluate` uses provided trusted metric code, not your incomplete metric function. Training selects the checkpoint with highest Val house-macro mIoU and saves it under `student_submission/weights/`.

6 Validation evidence and interpretation

Run full validation on the final method and generate report figures. Show the three input views, observed context, eligible hidden GT, and predictions from the same sample and viewpoint. Include successes and failures, per-class IoU and at least one controlled comparison. A high occupied IoU with lower semantic mIoU indicates good geometry but incorrect category assignments. Accuracy can be misleading when one class dominates; do not substitute it for the ranking metric.

The required figures are `inputs.png`, `completion.png`, `comparison.png`, and `class_iou.png` under `outputs/report/`. They demonstrate the system on provided simulator-generated data; no simulator installation or generated video is required. Additional figures and experiments are welcome. Report all full-Val metrics, not only selected favorable examples.

7 Further improve with your own algorithm

You may replace the architecture, loss, fusion method, representation, or entire algorithm. Completing unused baseline functions is not required for a deliberately different method. The runnable `alternative_method_template/` contains an all-free placeholder, not a competitive solution. It shows the exact input-loading and output interface without imposing a network design:

```
load_model(model_dir, device)
predict_sample(model, inputs, device)  # int [40,14,80], labels 0..9
```

The provided generator owns data loading, sample ordering, row IDs and CSV encoding. `load_model` runs once and receives the method's `weights/` directory, not its root. Inputs are NumPy float32 arrays: RGB `[3,3,H,W]` in $[0,1]$; depth `[3,1,H,W]` in metres (zero invalid), or `None`; intrinsics `[3,3]`; camera poses `[3,4,4]`; and anchor pose `[4,4]`. The CPU interface probe must pass. Do not pool other samples

or use house-specific rules. Your internal grid may differ, but convert its predictions to the fixed canonical grid before returning.

To start this path, copy the template's adapter and method declaration into `student_submission/`, add your implementation/dependencies, and keep required weights in its `weights/` directory. Declare your identity, algorithm, pretrained sources and external data in `method.json`. Public pretrained weights and external unlabeled data are allowed when declared and cited; external labeled semantic or occupancy datasets are prohibited. Keep the final code/model directory within 500 MiB.

```
python run.py validate                    # real fixed-grid CPU probe
python run.py evaluate --max-samples 3    # partial smoke evidence only
python run.py evaluate                    # required full Val evidence
```

Use Val for method and checkpoint selection. Explain controlled changes and limitations. The provided figure command works with any canonical-grid method. A replacement method may draw its own four required figures; record them with `python run.py figures --record-existing` after verifying they represent the final Val evaluation. Keep helper code under your method directory, preferably in a uniquely named package; do not shadow `cs498_track1/`. Baseline-specific architecture tests apply to the completed baseline, not a deliberately replaced architecture.

8 Kaggle challenge

After choosing your final method, download the input-only Test split:

```
python run.py download --split test
python run.py predict --max-samples 3 \
  --output outputs/kaggle/smoke.csv
python run.py predict
python run.py submit outputs/kaggle/submission.csv
python run.py submit outputs/kaggle/submission.csv --submit
```

The partial smoke CSV is not accepted by Kaggle. The full CSV has exactly 150 ordered rows, one per Test sample, with columns `Id,Prediction`. Row IDs have the form `test_house_000__sample_00000`. Each prediction is a full `int8 (z,y,x)` grid, flattened in C order, zlib-compressed and Base64-encoded. This encoding is formatting, not encryption. Use the provided generator; do not edit encoded predictions manually.

The competition is private. First open the course invitation `https://www.kaggle.com/t/f04e3dce46f047c5b52deec9fbe73701` while signed in to a Kaggle account whose verified email ends in `@illinois.edu`; before you join, the competition page shows a 404, which is expected. Then open `https://www.kaggle.com/competitions/cs498-fall-2026-multiview-occupancy`, accept the rules, and use a one-person team (teams are limited to one member) named `cs498-YOUR_NETID`. Kaggle receives predictions only and scores against private GT. The metric is house-macro completion mIoU over all ten houses; every row belongs to one live ranking partition. Runtime is recorded locally from method import and model loading through the complete CSV write; it only breaks an exact score tie after reproduction audit on the same hardware/software.

Do not manually annotate Test data, derive hidden labels from simulator assets, use sample-specific lookup predictions, share predictions, or tune via leaderboard probing. Every credited entry must be reproducible from the final code/model package. Authentication and HTTPS protect submission identity and transport; neither Base64 nor a student-created signature proves model provenance. Exact code/model hashes, data receipts and reproduction audits provide that check.

9 Report and final checks

Finalize `method.json`, code and weights before full Val evaluation and Test prediction. Any subsequent change requires rerunning both. Generate the evidence, complete the report template and package:

```
python run.py figures
python run.py report
python run.py package --student-id YOUR_NETID \
  --kaggle-username YOUR_KAGGLE_NAME
```

The report covers related work, design rationale, exact environment and commands, complete Val metrics, controlled comparisons, qualitative error analysis, the selected Kaggle entry and limitations. Record the full CSV and submission-tree SHA-256 values from `submission_runtime.json`. The ZIP's own digest is stored separately in `package_manifest.json`; upload both LMS artifacts. A placeholder report is not complete written evidence.

The packager verifies full 150-sample Val/Test coverage, matching source and CSV hashes, pinned data receipts, the CPU interface, required report, and a figure manifest tied to the final method and Val summary. Rerunning evaluation requires regenerating figures and rebuilding the report. These receipts prevent accidental stale evidence; they are not proof against fabricated results. Check your archive before submission. Never package data, credentials or hidden targets. For commands and troubleshooting, see `README.md` and `KAGGLE_SUBMISSION.md`.

References

[1] M. Deitke et al. ProcTHOR: Large-Scale Embodied AI Using Procedural Generation. NeurIPS, 2022.

[2] E. Kolve et al. AI2-THOR: An Interactive 3D Environment for Visual AI. arXiv:1712.05474, 2017.

[3] A. Kar et al. Learning a Multi-View Stereo Machine. NeurIPS, 2017.

[4] O. Çiçek et al. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. MICCAI, 2016.